Embeddings & Vector Similarity Search

Practical Learning Notes & Interview Preparation Guide

1. Concept Purpose

What problem does this concept solve? Computers cannot natively understand the semantic meaning of natural language text. Exact keyword search (e.g. SQL `LIKE %term%` or regex) fails when users use synonyms, paraphrasing, typos, or different languages (e.g. searching "automobile" will not match "car", even though their meaning is identical).

Why do we need it? Embedding models map arbitrary textual passages into dense numerical vectors in a continuous multi-dimensional geometric space. In this vector space, geometric proximity corresponds directly to semantic similarity: sentences with similar meanings cluster together regardless of lexical differences.

Where is it used? The backbone of Retrieval-Augmented Generation (RAG), semantic document search, recommendation engines, clustering, duplicate detection, and anomaly detection.

Why did you use it in your implementation? In your `EmbeddedModels/` directory, you built:

- Single-query vector generation using OpenAI models via OpenRouter (`embedding_openai_query.py`).
- Batch document vector generation (`embedding_openai_docx.py`).
- Zero-cost, local offline embedding generation using Hugging Face's `sentence-transformers/all-MiniLM-L6-v2` (`embedding_hf_local.py`).
- A complete semantic document search engine comparing a natural language query against sports player descriptions using Cosine Similarity (`document_similarity.py`).

2. Code Structure & Deep Analysis of Your Repository Files

File 1: `EmbeddedModels/embedding_openai_query.py`

```
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
import os

load_dotenv()

embedding = OpenAIEmbeddings(
    model="openai/text-embedding-3-small",
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENAI_API_KEY")
)

result = embedding.embed_query("Delhi is the capital of India", dimensions=32)
print("Vector dimensions:", len(result))
print(str(result))
```

Component Breakdown:

- `OpenAIEmbeddings`: LangChain's embedding integration implementing `Embeddings` interface.
- `model="openai/text-embedding-3-small"`: OpenAI's 3rd-generation high-efficiency embedding model (standard output: 1536 float dimensions).
- `embed_query(text)`: The core method to vectorize a single query string. It returns a `List[float]` representing the

```
vector coordinates in semantic space.
```

File 2: `EmbeddedModels/embedding_openai_docx.py`

```
documents = [
    "Delhi is the Capital of India",
    "Kolkata is the Capital of West Bengal",
    "Paris is the capital of france"
]
result = embedding.embed_documents(documents)
print("Vector dimensions:", len(result))
```

Component Breakdown:

- `embed_documents(List[str])`: Batches multiple document chunks into a single API request, reducing network latency. It returns a list of lists of floats: `List[List[float]]`.
- *Note on your code:* `len(result)` evaluates to `3` (the document count), while `len(result[0])` gives the actual vector dimensionality (e.g. 1536).

File 3: `EmbeddedModels/embedding_hf_local.py`

```
from langchain_huggingface import HuggingFaceEmbeddings

embedding = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
text = "Delhi is the capital is India"
vector = embedding.embed_query(text)
print(str(vector))
```

Component Breakdown:

- `HuggingFaceEmbeddings`: Uses local PyTorch and Hugging Face `transformers`/`sentence_transformers` pipelines.
- `model_name="sentence-transformers/all-MiniLM-L6-v2"`: A 384-dimensional model optimized for semantic search that runs entirely on your local machine with **zero API cost, zero network latency, and complete privacy**.

File 4: `EmbeddedModels/document_similarity.py`

```
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv
from sklearn.metrics.pairwise import cosine_similarity

load_dotenv()
embedding = OpenAIEmbeddings(
    model="openai/text-embedding-3-small",
    dimensions=300,
    base_url="https://openrouter.ai/api/v1"
)

documents = [
    "Virat Kohli is an Indian cricketer known for his aggressive batting and consistency in run chase
    "Rohit Sharma is an Indian opening batsman famous for his elegant stroke play...",
    "MS Dhoni is a former Indian captain known for his wicketkeeping skills...",
    "Jasprit Bumrah is an Indian fast bowler recognized for his deadly yorkers...",
    "AB de Villiers is a former South African cricketer admired for his innovative shots..."
]

query = "tell me about virat kohli"
```

```
doc_embeddings = embedding.embed_documents(documents)
query_embedding = embedding.embed_query(query)

result = cosine_similarity([query_embedding], doc_embeddings)[0]
index, score = sorted(list(enumerate(result)), key=lambda x: x[1])[-1]

print(query)
print(documents[index])
print("similarity score is: ", score)
```

Component Breakdown:

- `cosine_similarity([query_embedding], doc_embeddings)`: Calculates the cosine of the angle between the 1D query vector and the 2D array of 5 document vectors. Returns an array of scores between -1.0 and 1.0 (typically 0.0 to 1.0 for normalized embeddings).
- `sorted(..., key=lambda x: x[1])[-1]`: Pairs each score with its document index via `enumerate()`, sorts ascending, and selects the last item (highest similarity).

3. Internal Execution Flow & Mathematical Mechanics

How does text become a semantic coordinate?

1. **Subword Tokenization:** The input text is split into subword tokens (e.g. WordPiece or BPE) and converted to token IDs.
2. **Transformer Forward Pass:** Tokens pass through bidirectional self-attention transformer layers, producing a sequence of contextual hidden state vectors.
3. **Pooling Layer:** Mean-pooling or CLS-token pooling aggregates all token states into a single fixed-length dense vector.
4. **L2 Normalization:** The resulting vector v is normalized such that its Euclidean length $\|v\|_2 = 1.0$.
5. **Cosine Similarity Calculation:** The cosine of the angle $ heta$ between query vector Q and document vector D is defined as:
   $$\cos(heta) = rac{Q \cdot D}{\|Q\|_2 \|D\|_2}$$
   Because modern embedding models produce unit-normalized vectors ($\|Q\| = 1, \|D\| = 1$), Cosine Similarity simplifies to a pure **Dot Product**:
   $$\cos(heta) = \sum_{i=1}^{d} Q_i \cdot D_i$$ This dot product operation runs in microseconds using BLAS CPU vectorization (or GPU matrix multiplication).

4. Workflow Diagram

```
[ Corpus of 5 Player Bios ] [ User Query: "tell me about virat kohli" ] | | v v
.embed_documents(documents) .embed_query(query) | | v v 2D Matrix: (5 docs x 300 dims) 1D Vector: (1 x
300 dims) | | +----------------------+--------------------------+ | v scikit-learn:
cosine_similarity([Q], Doc_Matrix) | v [0.78, 0.45, 0.42, 0.38, 0.35] | v Argmax / Sort by Highest
Score | v Match: Virat Kohli Bio (Similarity Score: 0.78)
```

5. Recommended Production Architecture (Vector Store vs In-Memory Scikit-Learn)

In-memory cosine similarity via scikit-learn works great for 5 documents, but does not scale to millions of documents ($O(N)$ linear scan becomes sluggish and memory-intensive). In production, you replace manual matrix calculation with a Vector Database (FAISS, Chroma, Pinecone, or PGVector) that uses Approximate Nearest Neighbor (ANN) index structures like HNSW (Hierarchical Navigable Small World).

```
semantic_search_engine/
│
├── config/
│   └── settings.py
│
├── indexing/
│   ├── __init__.py
│   ├── document_loader.py    # Loads PDFs, text, Markdown
│   ├── text_splitter.py      # Chunks large files
│   └── indexer.py            # Vectorizes & saves to FAISS / Chroma
│
├── search/
│   ├── __init__.py
│   └── retriever.py          # Semantic similarity & MMR ranking
│
├── data/
│   ├── raw_documents/
│   └── faiss_index/          # Persisted vector database
│
└── main.py
```

6. Implementation From Scratch (Step-by-Step Exercise)

Objective: Build a local, offline Semantic Search engine using `HuggingFaceEmbeddings` and LangChain's in-memory FAISS vector store.

```
# Complete Reference Solution
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document

# 1. Initialize local embedding model (Zero API Cost)
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

# 2. Prepare structured documents
docs = [
    Document(page_content="Python is an interpreted, high-level programming language created by Guido
    Document(page_content="Docker automates the deployment of applications inside lightweight containe
    Document(page_content="PostgreSQL is a powerful, open-source object-relational database system.",
    Document(page_content="Git is a distributed version control system for tracking changes in source
]

# 3. Create FAISS Vector Store Index
vectorstore = FAISS.from_documents(docs, embeddings)

# 4. Perform Similarity Search with Score
query = "What tool should I use to manage code versions?"
results = vectorstore.similarity_search_with_score(query, k=2)

print(f"Query: {query}\n")
for doc, score in results:
    # Lower L2 distance = higher similarity
    print(f"[Score: {score:.4f}] {doc.page_content} (Source: {doc.metadata['source']})")
```

7. Common Errors & Actual Repository Bugs

ERROR 1: Passing `dimensions` as a query parameter (From EmbeddedModels/embedding_openai_query.py)

Your Code:

```
result = embedding.embed_query("Delhi is the capital of India", dimensions=32)
```

Why It Fails: In LangChain's `OpenAIEmbeddings`, `dimensions` is a constructor configuration parameter for OpenAI's `text-embedding-3` models (e.g. `OpenAIEmbeddings(model="...", dimensions=32)`), NOT a keyword argument accepted by `.embed_query(text)`! Passing it to `embed_query()` either causes a `TypeError: unexpected keyword argument` or is silently ignored.

Correct Fix: Define dimensions during class initialization:

```
embedding = OpenAIEmbeddings(
    model="openai/text-embedding-3-small",
    dimensions=32,
    base_url="https://openrouter.ai/api/v1"
)
result = embedding.embed_query("Delhi is the capital of India")
```

ERROR 2: Misinterpreting Document Array Length (From EmbeddedModels/embedding_openai_docx.py)

Your Code:

```
result = embedding.embed_documents(documents)
print("Vector dimensions:", len(result))
```

Why It Is Confusing: `result` is a list of vectors (`List[List[float]]`). `len(result)` equals `3` (the count of documents embedded), NOT the vector dimension! To print the embedding vector dimensionality, you must check `len(result[0])`.

8. Explain It to Someone Else (Speaking Frameworks)

- **Simple Explanation:** "Embeddings are GPS coordinates for ideas. Just as coordinates with similar latitudes and longitudes are geographically close on a map, words and sentences with similar meanings are placed right next to each other in a mathematical coordinate space."
- **Technical Explanation:** "An embedding model maps discrete linguistic tokens into a dense, continuous latent vector space $\mathbb{R}^d$. The resulting vectors encapsulate semantic, syntactic, and contextual relationships such that inner product or cosine distance correlates with semantic similarity."
- **60-Second Interview Explanation:** "In modern GenAI architectures, embeddings are the foundation of semantic retrieval and RAG. They convert text chunks into fixed-dimensional vectors where similarity can be computed mathematically using Cosine Similarity or Dot Product. In my projects, I evaluated both cloud embeddings like OpenAI's `text-embedding-3-small` and local sentence-transformers like `all-MiniLM-L6-v2`. I implemented cosine ranking over candidate documents, which provides the foundational mechanics before moving to production vector indexes like FAISS."
- **Whiteboard Explanation:** Draw a 2D coordinate graph with axes X and Y. Plot three points: "King" $(8, 9)$, "Queen" $(8, 8.8)$, and "Apple" $(1, 2)$. Draw vectors from origin $(0,0)$. Show that the angle between "King" and "Queen" is tiny ($\cos heta ≈ 0.95$), while the angle to "Apple" is large ($\cos heta ≈ 0.1$).
- **Real-World Analogy:** Color mixing in RGB. A computer cannot understand "warm orange" or "golden yellow" as words, but it represents them as `(255, 140, 0)` and `(255, 215, 0)`. Because their RGB coordinates are close, software knows they are visually similar.

9. Interview Preparation Questions & Answers

Level	Question	Technically Exact Answer
Basic	What is the difference between `embed_query()` and `embed_documents()`?	`embed_query()` takes a single string and returns a 1D vector (`List[float]`), designed for runtime search queries. `embed_documents()` takes a list of strings and returns a 2D matrix (`List[List[float]]`), optimized for batching multiple chunks during indexing. Some asymmetric models (like Cohere or E5) apply different prompt prefixes for queries versus documents.
Basic	Why is Cosine Similarity preferred over Euclidean (L2) distance for text embeddings?	Cosine similarity measures the angle between vectors, making it invariant to vector magnitude (text length). Two passages covering the exact same topic—one being a short tweet and the other a long paragraph—will point in the same direction, giving a high cosine similarity even if their Euclidean distance is large.
Intermediate	What are Matryoshka Embeddings (supported by OpenAI text-embedding-3)?	Matryoshka Representation Learning trains models so that early dimensions (e.g. the first 256 or 512 dimensions of a 1536-dim vector) retain most of the semantic fidelity. This allows developers to truncate vectors to smaller sizes, dramatically reducing vector database storage costs and latency with minimal accuracy loss.
Intermediate	What is the trade-off between local embedding models and cloud API models?	Local models (e.g. `all-MiniLM-L6-v2`, `bge-small`) offer zero API cost, zero network latency, complete data privacy, and offline operation, but require local compute/RAM. Cloud APIs (OpenAI, Cohere) require no local GPU compute and handle massive multilinguality, but incur recurring costs and network latency.
Advanced	What is the difference between Dense Retrieval and Sparse Retrieval (BM25)?	Dense retrieval (embeddings) captures semantic context and synonyms but can struggle with exact keyword matches, serial numbers, or acronyms. Sparse retrieval (BM25/TF-IDF) matches exact tokens with term-frequency weighting. Production systems combine both using Hybrid Search with Reciprocal Rank Fusion (RRF).
Advanced	What is Maximum Marginal Relevance (MMR) and why is it used in retrievers?	Standard similarity search often returns k nearly identical chunks. MMR optimizes for both relevance to the query AND diversity among selected documents, preventing redundant chunks from clogging the LLM context window.